



Robotics & Computer Vision

Inverse Kinematics: Manipulators & Mobile Robots

Week 8 — From joint/wheel inputs to end-effector and base pose — the foundational mapping every roboticist must master.

Lê Đình Phong, Ph.D

Week 8 Learning Objectives

1

Understand IK Problem

From a target pose $\rightarrow$ find the joint configuration that achieves it

2

Analytic vs. Numerical IK

Know the difference, trade-offs, and when to apply each approach.

3

Use `ikine` / `ikine_LM`

Apply Robotics Toolbox functions to find joint angles for a manipulator.

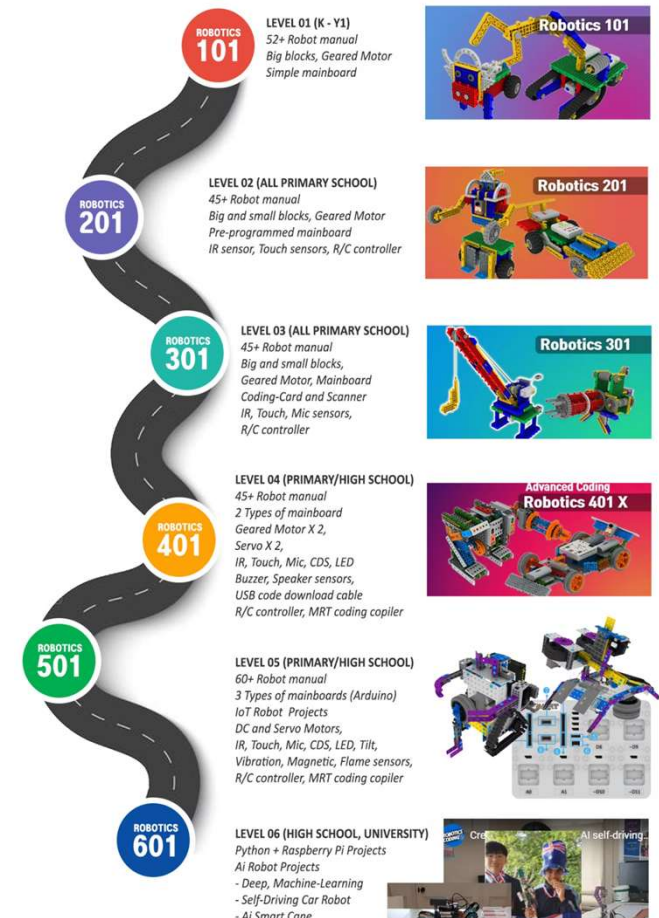
4

Mobile Robot "IK"

Translate desired trajectory into wheel commands (v_L , v_R).

Connection to Weeks 6–9

- 1 — Week 6
SE(3), Product of Exponentials, Jacobian — general kinematic framework.
- 2 — Week 7
Forward Kinematics — from joint angles / wheel speeds → end-effector pose.
- 3 — Week 8
Inverse Kinematics — from desired pose → joint angles / wheel commands.
- 4 — Week 9
Vision-guided grasping — AI/vision → T_{target} → IK → robot picks object.



Why Is IK So Important?

Task Space

Vision systems and motion planners operate in **task space** — they produce a desired 6D pose (position + orientation) for the end-effector.

Joint Space

Robot servo controllers only accept **joint commands** — motor angles or velocities for each individual joint.

IK Is the Bridge

Inverse Kinematics translates perception/planning outputs into actionable joint commands. Without IK, there is no path from "seeing" to "doing."



Formal Definition of IK

Given a desired end-effector pose $T_{\text{des}} \in SE(3)$, find the joint vector $\mathbf{q}$ such that $F(\mathbf{q}) \approx T_{\text{des}}$, where F is the Forward Kinematics function.

Forward Kinematics (FK)

Always **unique**: given a joint configuration $\mathbf{q}$, there is exactly one resulting pose. If the model is correct, FK never fails.

Inverse Kinematics (IK)

Can have **many solutions**, a single solution, or **no solution** at all — depending on workspace, joint limits, and robot geometry.

❗ IK uses FK as its internal "check": compute $F(\mathbf{q})$ and measure how close it is to T_{des} .

Key Properties of IK Solutions

Non-Uniqueness

A single Tdes can correspond to multiple joint configurations — e.g., elbow-up vs. elbow-down, or up to 16 solutions for a general 6-DOF arm.

Non-Existence

If the target lies outside the robot's reachable workspace, or violates joint limits, no valid solution exists. The IK solver must detect and report this.

Constraint Dependency

The preferred solution depends on joint limits, collision avoidance, singularity avoidance, and proximity to the current configuration — not just kinematics.

When There Is No Solution

Outside Workspace

The target position is physically beyond the robot's maximum reach. No joint configuration can place the TCP there.

Orientation Infeasible

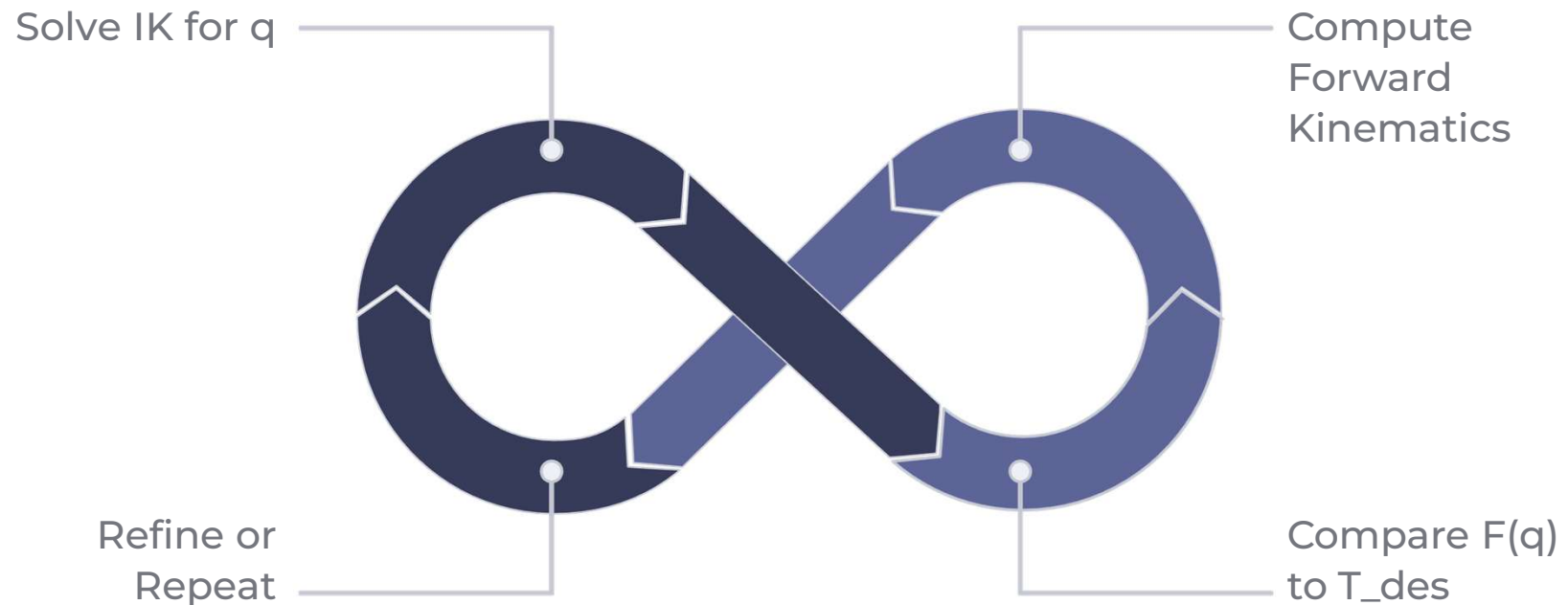
The required wrist orientation exceeds the wrist joint's range of motion, even if position is reachable.

Joint Limits

A target that is theoretically reachable becomes infeasible when joint limits are applied. Always model joint limits correctly.

 Always check the **sol.success** flag after calling `ikine_LM` — a failed IK should never be silently passed to the controller.

FK as the IK Verification Tool



After obtaining a joint solution q , always compute $F(q)$ with Forward Kinematics and measure the pose error. If the position error exceeds a few millimeters or rotation error exceeds a fraction of a degree, the solution is not acceptable. This **FK-IK consistency check** is a core habit practiced in Lab 8.3.

Analytic vs. Numerical IK

Analytic (Closed-Form)

- Uses geometry and trigonometry to derive explicit formulas
- Extremely fast — single computation, no iteration
- Gives all solutions simultaneously
- Only feasible for special structures: PUMA, 2R, 3R, spherical wrist

Numerical (Iterative)

- Iteratively minimizes pose error using the Jacobian
- Fully general — works for any robot geometry
- Result depends on initial guess q_0
- May converge slowly or get stuck in local minima

Analytic IK on 2R/3R

Build geometric intuition. Understand why multiple solutions exist and how to pick one.

Numerical IK via `ikine_LM`

Practical tool for 6-DOF arms (Panda, Yahboom). No manual derivation required.

Full Pipeline Focus

Perception → Target Pose → IK → Robot Move. IK is one piece of a larger system.

Students are not expected to derive IK formulas for a 6-DOF arm by hand. The emphasis is on understanding what IK does and integrating it correctly into the robotics pipeline.

Analytic IK for the 2R Planar Robot — Model

Model Parameters

- Link lengths: l_1, l_2
- Joint angles: θ_1, θ_2
- End-effector position: (x, y)

Forward Kinematics

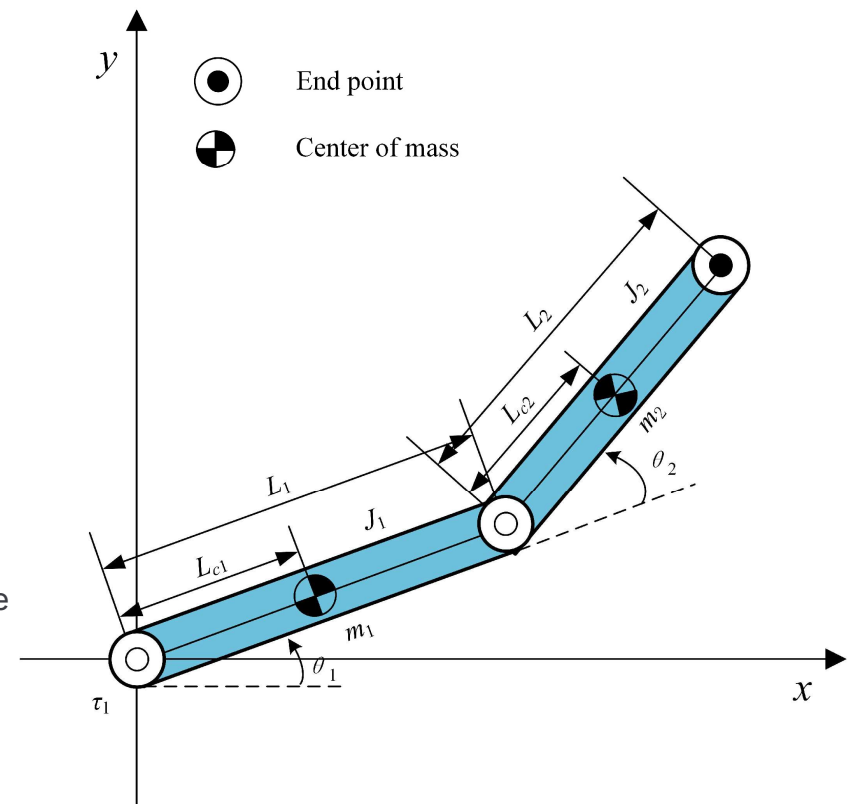
$$x = l_1 \cos(\theta_1) + l_2 \cos(\theta_1 + \theta_2)$$

$$y = l_1 \sin(\theta_1) + l_2 \sin(\theta_1 + \theta_2)$$

IK Problem

Given desired target (x_d, y_d) , find joint angles (θ_1, θ_2) that position the TCP exactly at that point.

This is the simplest non-trivial IK problem and already reveals all the key challenges: multiple solutions, singularities, and workspace boundaries.



The 2R IK Problem Setup

Given

- Link lengths: l_1, l_2
- Target point: (x_d, y_d) in the plane

Find

- Joint angles: θ_1, θ_2
- Such that the TCP reaches (x_d, y_d)

Key Insight: Two Solutions

For any reachable point inside the annular workspace (where $|l_1 - l_2| \leq r \leq l_1 + l_2$), there exist two valid joint configurations:

- **Elbow-up:** $\theta_2 > 0$
- **Elbow-down:** $\theta_2 < 0$

At the workspace boundary, the two solutions merge into one.

Solving Analytic IK for 2R

1 Compute Distance to Target

Calculate $r = \sqrt{x_d^2 + y_d^2}$, the straight-line distance from the robot base to the target point.

2 Find θ_2 Using the Law of Cosines

$\cos(\theta_2) = (r^2 - l_1^2 - l_2^2) / (2l_1l_2)$. If $|\cos(\theta_2)| > 1$, the point is outside the workspace. The $\pm$ gives elbow-up / elbow-down.

3 Find θ_1 from Triangle Geometry

$\theta_1 = \text{atan2}(y_d, x_d) - \text{atan2}(l_2 \sin \theta_2, l_1 + l_2 \cos \theta_2)$. Subtract the angle subtended by the second link.

Lessons from the 2R Example



IK Is Geometry-Dependent

Every formula is specific to the robot's link structure. A different robot means re-deriving from scratch.



Multiple Solutions Are the Rule

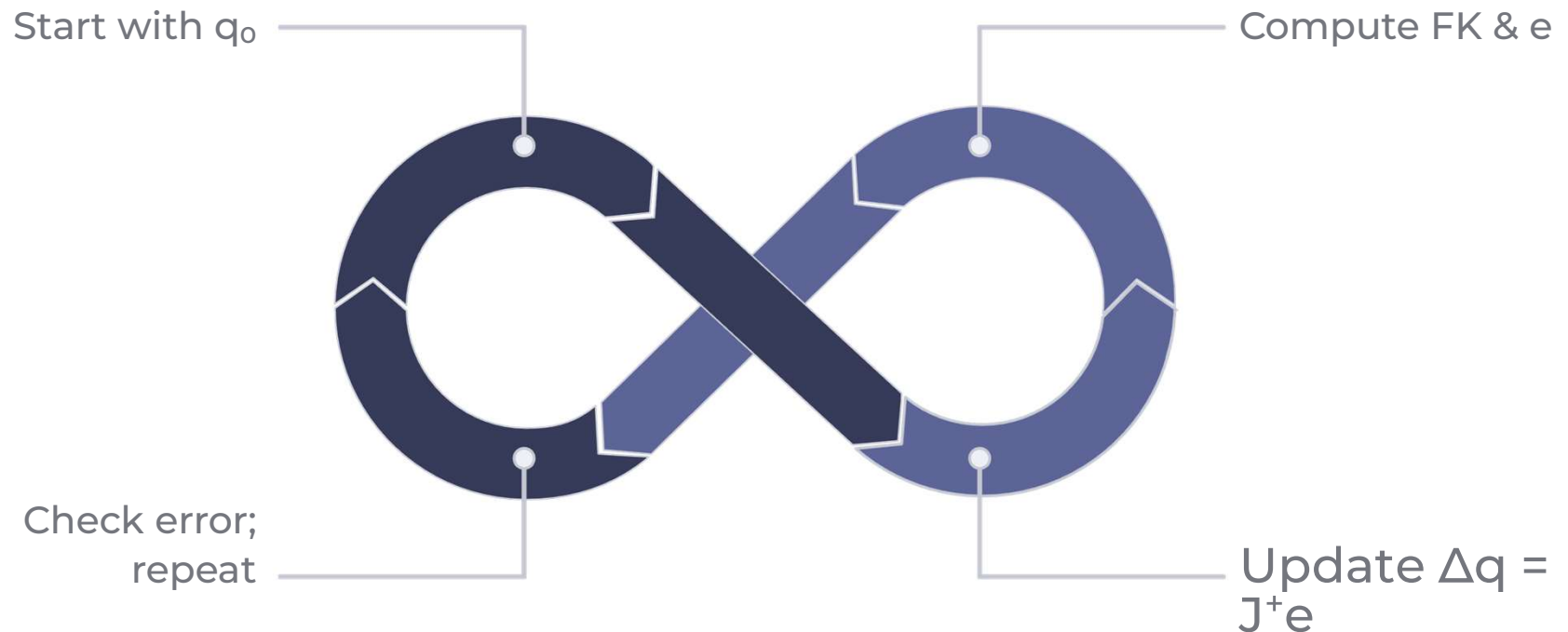
Even a simple 2R robot has two solutions and a no-solution zone. These concepts scale to 6-DOF.



Complexity Explodes at 6-DOF

For an arbitrary 6-DOF arm, analytic IK involves systems of transcendental equations — hence we use numerical IK in practice.

Numerical IK & Jacobian - Based Methods



Numerical IK is fundamentally an optimization loop. Starting from an initial guess q_0 , it repeatedly uses the Jacobian to compute a correction Δq that reduces the pose error. The loop exits when the error falls below an acceptable threshold — typically millimeter-level position and milliradians orientation.

Pose Error and Joint Update

Pose Error $\mathbf{e}$

The error is computed in SE(3): $\mathbf{e} = \log(\mathbf{T}_{\text{cur}}^{-1} \cdot \mathbf{T}_{\text{des}})$, which gives a 6-vector capturing both position mismatch (3D) and orientation mismatch (3D axis-angle).

Jacobian-Based Update

The Jacobian $\mathbf{J}$ relates joint velocities $\dot{\mathbf{q}}$ to end-effector velocity $\dot{\mathbf{x}}$. Using the pseudo-inverse:

$$\Delta \mathbf{q} = \mathbf{J}^+ \cdot \mathbf{e}$$

where $\mathbf{J}^+ = \mathbf{J}^T(\mathbf{J}\mathbf{J}^T)^{-1}$ is the Moore-Penrose pseudo-inverse. This is valid away from singularities.

Damped Least Squares — Levenberg–Marquardt

Near singular configurations, $J^T J$ approaches singularity, causing the pseudoinverse update to become enormous and unstable. The Levenberg–Marquardt (LM) method adds a damping term:

$$\Delta q = (J^T J + \lambda^2 I)^{-1} J^T e$$

Effect of Damping λ

- $\lambda \rightarrow 0$: Recovers pseudoinverse (fast convergence away from singularities)
- λ **large**: Gradient-descent-like step (stable near singularities, slower)
- Adaptive λ : increase near singularities, decrease elsewhere

In Robotics Toolbox

`robot.ikine_LM()` implements a variant of DLS/LM. It automatically manages damping to balance convergence speed and numerical stability, making it robust for 6-DOF arm IK in both simulation and hardware contexts.

Impact of Initial Guess q_0

Good q_0 (Near Solution)

The algorithm starts in a region where the cost function is well-behaved. It converges quickly, in few iterations, to the desired solution — typically the one nearest to the current robot pose.

Poor q_0 (Far from Solution)

Convergence is slow, and the optimizer may settle into a different local minimum (a different IK solution), or fail to converge entirely if the starting point is near a singularity.

Best Practice

Always use the **current robot configuration q_{cur}** as the initial guess. This produces smooth, predictable motion by selecting the IK solution closest to where the robot already is.

ikine_LM in Robotics Toolbox

Function Overview

`robot.ikine_LM(T_des, q0=...)` solves 6-DOF IK using the Levenberg–Marquardt method. It is the recommended numerical IK solver for the Robotics Toolbox `Robot` class.

Return Values

- `sol.q` — solution joint vector
- `sol.success` — True/False convergence flag
- `sol.iterations` — number of iterations used
- `sol.residual` — final pose error magnitude

Always Check `sol.success`

- ✗ If `sol.success` is False, do NOT use `sol.q`. The solver did not converge — check if the target is in the workspace, adjust `q0`, or increase max iterations.

After a successful solve, verify with FK: compute `T_check = robot.fkine(sol.q)` and confirm the position and orientation errors are within tolerance.

Numerical IK Example — Panda 6-DOF

```
import roboticstoolbox as rtb
from spatialmath import SE3
import numpy as np

# Load robot model
robot = rtb.models.URDF.Panda()

# Define desired end-effector pose
T_des = SE3(0.5, 0.0, 0.3) * SE3.Rx(np.deg2rad(90))

# Solve IK using Levenberg-Marquardt
sol = robot.ikine_LM(T_des, q0=robot.qr)
q_sol = sol.q
print("success =", sol.success,
      "iterations =", sol.iterations,
      "residual =", sol.residual)

# Verify with FK
T_check = robot.fkine(q_sol)
print("T_check:\n", T_check.A)
print("position error:", T_check.t - T_des.t)
```

 In lab: compare T_{des} and T_{check} side by side to validate the IK solution before sending to hardware.

IK for Yahboom 6-DOF

Course Specification

The course outline specifies: *"Use ikine functions to find joint angles when the target spatial coordinate of the arm is known."* The Yahboom 6-DOF model is described using ETS or URDF and shares the same Robotics Toolbox API as the Panda.

Full Pipeline

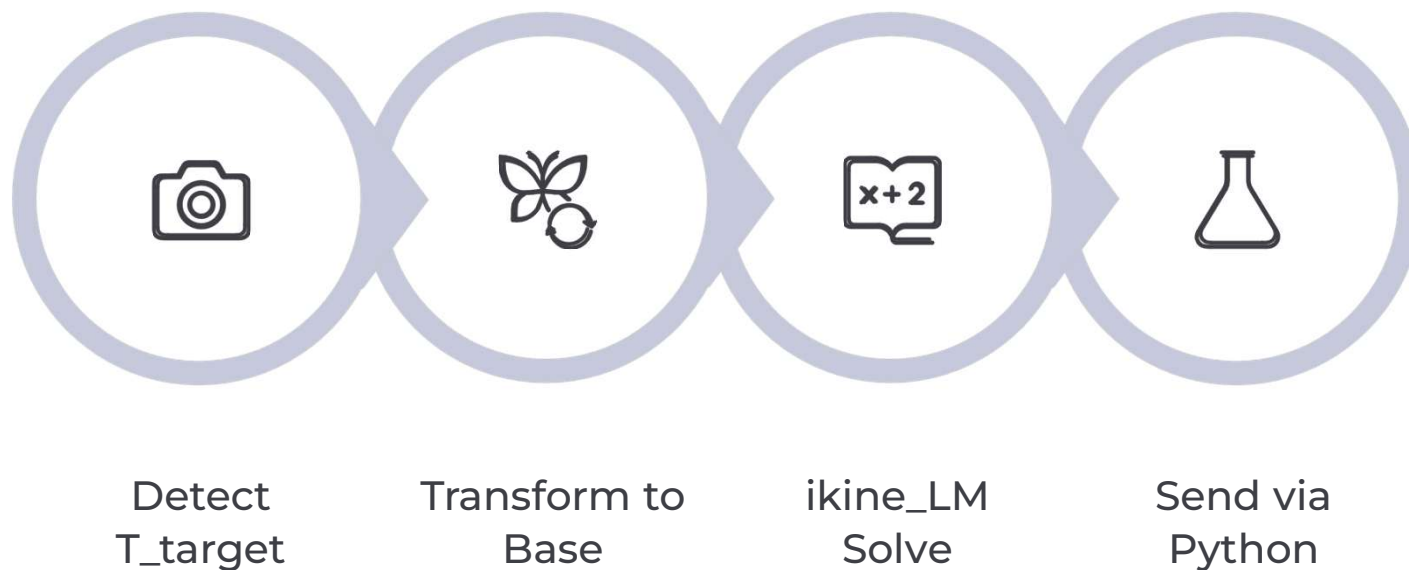


Integration Points

- **Input:** Target from AI pose estimation + coordinate mapping (Week 9)
- **IK Solver:** `ikine_LM` with current q as initial guess
- **Output:** joint angles sent via Python SDK to Yahboom servo controllers



IK for Yahboom in the RCV Pipeline



The Yahboom arm model can be declared using ETS (Elementary Transform Sequence) or loaded from a URDF file. Once the model is correct, **ikine_LM** works identically regardless of the physical robot. This is the engine that drives autonomous grasping in Week 9.

SECTION E

"IK" for Mobile Robots

The concept of Inverse Kinematics extends naturally to mobile robots. Instead of finding joint angles, the goal is to find **wheel velocity commands** (v_L , v_R) that produce a desired body motion (v , ω). This is the "IK" of the differential-drive platform.



Unicycle Mobile Robot Model — Review

State & Control

- State: (x, y, θ) — position and heading in the world frame
- Control inputs: v (forward speed), ω (angular rate)

Kinematic Model

$$\dot{x} = v \cos \theta, \quad \dot{y} = v \sin \theta, \quad \dot{\theta} = \omega$$

Interpretation

The unicycle model abstracts away the wheel mechanics and provides a clean Cartesian interface. The planner and controller work in (v, ω) space, which is then converted to actual wheel commands — exactly analogous to how arm planners work in task space before IK converts to joint space.

Differential Drive "IK" — $(v, \omega) \rightarrow (v_L, v_R)$

For a differential-drive robot with wheel track width W , the forward kinematics from wheel speeds to body motion is:

$$v = \frac{v_R + v_L}{2}, \quad \omega = \frac{v_R - v_L}{W}$$

Inverting these equations gives the "IK" of the mobile base — from desired (v, ω) to wheel commands:

$$v_R = v + \omega \frac{W}{2}, \quad v_L = v - \omega \frac{W}{2}$$

- ✓ This is a linear, closed-form inversion — no iteration needed. The mobile IK is always unique (unlike arm IK) as long as hardware limits are respected.

diff_drive_ik — Python Implementation

```
def diff_drive_ik(v, w, W):  
    """  
    Convert desired body velocity (v, w) to wheel speeds (v_L, v_R).  
  
    Args:  
        v : forward speed [m/s]  
        w : angular rate [rad/s]  
        W : wheel track width [m]  
  
    Returns:  
        v_L, v_R : left and right wheel speeds [m/s]  
    """  
    v_R = v + w * W / 2  
    v_L = v - w * W / 2  
    return v_L, v_R
```

This function is used in the mobile robot lab to compute wheel commands from a desired trajectory. It is also the final output stage in the line-following pipeline: **error** $\rightarrow \omega \rightarrow \text{diff_drive_ik} \rightarrow (v_L, v_R) \rightarrow \text{motors}$.

Line Following — From Image Error to ω

1

Camera Image

Detect lane centerline; measure pixel offset from image center

2

Error $\rightarrow \omega$

$\omega = k_e \cdot e$ (proportional controller); v held nearly constant

3

Mobile IK

$\text{diff_drive_ik}(v, \omega, W) \rightarrow v_L, v_R$

4

Robot Moves

Wheels receive commands; robot follows the line

Week 11 will expand this pipeline with full PID control and real camera input. This week, we simulate the error signal to understand the kinematics without the added complexity of vision.

The Big Picture: Arm IK vs. Mobile IK

Manipulator IK

- **Input:** 6D target pose T_{des} in $SE(3)$
- **Output:** joint angle vector q
- **Tool:** ikine_LM (Jacobian-based)
- **Check:** $FK(q) \approx T_{\text{des}}$

Mobile Robot IK

- **Input:** desired (v, ω) or curvature
- **Output:** wheel speeds (v_L, v_R)
- **Tool:** diff_drive_ik formula
- **Check:** simulate unicycle model

- ✓ Both problems are **inverse mappings** between the space where AI/vision operates (task/velocity space) and the space where actuators receive commands (joint/wheel space).

Week 8 Lab — Overview

This week's lab spans two major tracks: **manipulator IK** (analytic 2R + numerical 6-DOF) and **mobile robot "IK"** (wheel commands + line following). Labs are designed to build from first principles to full verification and hardware readiness.

01

Lab 8.1 — Analytic IK (2R)

Implement geometric IK from scratch. ~25 min.

02

Labs 8.2–8.4 — Numerical IK (6-DOF)

ikine_LM, workspace limits, convergence study. ~85 min total.

03

Labs 8.5–8.6 — Trajectory & Solution Selection

Interpolation, Swift/Yahboom execution, best-solution criteria.

04

Labs 8.M1–M4 — Mobile IK & Line Following

diff_drive_ik, simulation, saturation, real robot pipeline.

LAB 8.1

Analytic IK for the 2R Planar Robot

Objective

Students derive and implement the complete analytic IK for a 2R planar robot using the law of cosines and atan2 formula — no library calls, only geometry and NumPy.

Specification

- **Input:** $l_1, l_2, (x_d, y_d)$
- **Output:** 0, 1, or 2 solutions (θ_1, θ_2) with elbow classification
- **Time:** ~25 minutes

Deliverables

- Working `ik_2r(l1, l2, x, y)` function
- Test with reachable and unreachable targets
- Visualization: draw both arm configurations using matplotlib
- Verify each solution with FK and report position error

Lab 8.1 — Reference Implementation

```
import numpy as np

def ik_2r(l1, l2, x, y):
    """Returns list of (theta1, theta2) solutions."""
    r2 = x**2 + y**2
    cos_t2 = (r2 - l1**2 - l2**2) / (2 * l1 * l2)

    if abs(cos_t2) > 1.0:
        return [] # Target unreachable

    def compute_theta1(theta2):
        phi = np.arctan2(y, x)
        beta = np.arctan2(l2 * np.sin(theta2),
                          l1 + l2 * np.cos(theta2))
        return phi - beta

    t2_down = +np.arccos(cos_t2) # elbow-down
    t2_up = -np.arccos(cos_t2) # elbow-up

    sols = []
    for t2, label in [(t2_down, "elbow-down"),
                      (t2_up, "elbow-up")]:
        t1 = compute_theta1(t2)
        sols.append((t1, t2, label))
    return sols
```

 Exercise: plot both arm configurations on the same axes using matplotlib to visually confirm the two solutions reach the same (x_d, y_d).

LAB 8.2

Numerical IK for 6-DOF Arm (Panda / Yahboom)

Objective

Use `ikine_LM` to solve IK for 3–5 distinct target poses. Verify each solution using FK, compute and report position and orientation errors.

Task

- Select 3–5 simple `Tdes` poses (vary x , y , z and light rotations)
- Solve IK with `q0 = robot.qr`
- Print `Tdes`, `Tcheck`, position error vector
- Record `sol.success`, `sol.iterations`, `sol.residual`

Time: ~30 minutes

Code Template

```
targets = [  
    SE3(0.5, 0.0, 0.3),  
    SE3(0.4, 0.2, 0.25)  
    * SE3.Rz(np.deg2rad(30)),  
    SE3(0.45, -0.15, 0.28)  
]  
q0 = robot.qr  
for T_des in targets:  
    sol = robot.ikine_LM(T_des, q0=q0)  
    q_sol = sol.q  
    T_check = robot.fkine(q_sol)  
    err = T_check.t - T_des.t  
    print(sol.success,  
          sol.iterations, err)
```


LAB 8.2(B)

IK with Joint Limits & Workspace Boundary

Select Boundary Targets

Choose target poses **Tdes** near or outside the robot's reachable workspace — pushing the arm to its kinematic limits.

Observe Failures

Watch for `sol.success = False` or joint solutions `q_sol` that violate the robot's joint limits. Record which targets cause failure.

Workspace Visualization

Mark successful and failed targets on a 2D or 3D plot of the TCP workspace. Identify the reachable boundary visually.

- ❏ Understanding workspace limits is critical for real deployment — sending an infeasible IK target to hardware is a common source of robot faults.

LAB 8.2(C)

Selecting IK Solution Closest to Current Config

Experiment

For the same target T_{des} , solve IK twice:

- Run 1: $q_0 = \text{robot.qz}$ (zero configuration)
- Run 2: $q_0 = q_{cur}$ (current robot pose)

Compare the resulting q_{sol} vectors and compute $\|q_{sol} - q_{cur}\|$ for each run.

Key Takeaway

Using q_{cur} as the initial guess systematically selects the IK solution closest to the current configuration. This produces the **smallest joint displacement**, leading to smoother, more predictable robot motion — essential for real hardware safety.

- ✓ Best Practice: always set $q_0 = q_{cur}$ in operational code.

LAB 8.3

FK–IK Consistency Check

This lab verifies the mathematical consistency of FK and IK by testing the **round-trip property**: $FK(IK(FK(q))) \approx FK(q)$.

Step 1: Random Configs

Sample several joint configurations q_{random} uniformly within the robot's joint limits using `np.random.uniform`.

Step 2: FK $\rightarrow$ IK

Compute $T = \text{robot.fkine}(q_{\text{random}})$, then solve $q' = \text{ikine_LM}(T, q_0=q_{\text{random}})$. This should recover the original or equivalent configuration.

Step 3: Compare

Compute $\|q' - q_{\text{random}}\|$. For non-redundant robots this should be near zero. For redundant robots (7-DOF), small differences are expected.

Time: ~25 minutes

LAB 8.3(B)

Mini Trajectory Around a Target — IK-Planned

Setup

Choose a center pose T_0 in the middle of the workspace.
Generate a set of nearby poses:

- Translate by $\pm 1\text{--}2$ cm along x , y , z axes
- Rotate by $\pm 5^\circ$ around each axis

Solve IK for each pose to get joint configurations q_i .

Visualization

Compute FK for each q_i and plot the resulting TCP positions in 3D space. Observe how small task-space perturbations map to joint-space changes. This illustrates the local geometry of the FK/IK mapping and is a precursor to trajectory planning covered in later weeks.

LAB 8.4

Effect of Initial Guess on IK Convergence

Choose a target **Tdes** known to have multiple IK solutions. Run `ikine_LM` with different initial guesses and systematically compare results.

1

$q0 = \text{robot.qz}$

All-zero configuration. Record iterations, residual, and resulting q_{sol} .

2

$q0 = \text{robot.qr}$

Ready/nominal configuration. Often provides better convergence for typical workspace targets.

3

$q0 = \text{random \#1}$

Randomly sampled within joint limits. Result is unpredictable — may converge to different solution.

4

$q0 = \text{random \#2}$

Another random sample. Compare which initial guesses consistently reach the desired solution.

Convergence Comparison Table

Summarize all runs in a structured table to identify patterns in how initial guess quality affects numerical IK performance:

Initial Guess q_0	Iterations	Residual	Notes
robot.qz (zero)	—	—	Record from your run
robot.qr (ready)	—	—	Often best for typical targets
q_random #1	—	—	May find different solution
q_random #2	—	—	Compare q_sol difference
q_cur (current)	—	—	Best for smooth motion

 Discussion: Why does a q_0 closer to the true solution lead to fewer iterations? Relate to the shape of the cost function landscape.

LAB 8.5

From IK to Real Motion — Swift / Yahboom

Objective

Close the loop between IK computation and physical (or simulated) robot execution. This is the most complete lab exercise of the week, combining all previous skills.

Procedure

1. Select a target T_{des} within the workspace
2. Read or set q_{cur} as the current configuration
3. Solve IK: $q_{des} = \text{ikine_LM}(T_{des}, q_0=q_{cur}).q$
4. Interpolate: $q(t) = (1-t)*q_{cur} + t*q_{des}$ for $t \in [0,1]$
5. Execute trajectory on Swift simulator or Yahboom hardware
6. Plot TCP path in 3D and verify it reaches T_{des}

Key Observation

Linear interpolation in joint space does **not** produce a straight-line TCP path in task space. This motivates task-space trajectory planning (Cartesian interpolation), covered in later weeks.

LAB 8.6

Synthesis — Selecting the Best IK Solution

When multiple IK solutions exist for the same T_{des} (analytic or numerical with different q_0), we need a principled way to choose the best one for the robot's current state and constraints.



Proximity to q_{cur}

Minimize $\|q_{sol} - q_{cur}\|$ to ensure the smallest joint motion and smoothest trajectory.



Joint Limit Compliance

Reject any solution where any joint angle exceeds its hardware limit. Filter candidates before scoring.



Singularity Avoidance

Compute $\det(J \cdot J^T)$ or manipulability index. Prefer solutions where this is maximized.

Implement a `select_best_ik(solutions, q_cur, joint_limits)` function that returns the optimal solution according to these criteria.

LAB 8.M1

Mobile "IK" — From (v, ω) to (vL, vR)

Tasks

1. Implement `diff_drive_ik(v, w, W)`
2. Create a motion profile: constant v , constant ω
3. Simulate trajectory $(x(t), y(t), \theta(t))$ using the unicycle model
4. Try multiple ω values and observe varying turning radii

Turning Radius Formula

$$R_{turn} = \frac{v}{\omega}$$

Expected Observations

- $\omega = 0$: Straight line ($vL = vR$)
- ω **small**: Large turning radius (gentle curve)
- ω **large**: Small turning radius (tight circle)
- $\omega = v/r$: Circle of radius r
- $v = 0, \omega \neq 0$: Spin in place

LAB 8.M2

Line Following 1D Simulation

```
import numpy as np

def simulate_line_following(e_series, ke, v0, W, dt=0.1):
    """
    e_series : array of lateral errors [pixels]
    ke       : proportional gain [rad/s per pixel]
    v0       : constant forward speed [m/s]
    W        : wheel track width [m]
    """
    x, y, theta = 0.0, 0.0, 0.0
    trajectory = [(x, y, theta)]

    for e in e_series:
        omega = ke * e
        v_L, v_R = diff_drive_ik(v0, omega, W)
        # Unicycle step
        x += v0 * np.cos(theta) * dt
        y += v0 * np.sin(theta) * dt
        theta += omega * dt
        trajectory.append((x, y, theta))
    return trajectory
```

Test with: step error (sudden offset), sinusoidal error (winding path), and small random noise. Observe how the proportional controller drives the robot back toward the line.

LAB 8.M3

Saturation on v and ω in Line Following

Why Saturation Matters

Real motors have maximum speeds. If the controller produces $|\omega| > \omega_{\max}$ or $|v| > v_{\max}$, the command is clipped — introducing nonlinearity that affects stability and tracking quality

```
omega = np.clip(ke * e, -w_max, w_max)
v = np.clip(v0, -v_max, v_max)
```

Comparison Study

- **Without saturation:** Large errors cause violent steering — the robot may oscillate or overshoot
- **With saturation:** Motion is bounded but may be slower to correct large errors
- **Trade-off:** Stability vs. response speed vs. hardware constraints

□ Discuss: what happens if k_e is too large even with saturation?

LAB 8.M4

Connecting Simulation to Real JetRacer / TurtleBot

1

Camera

Captures image frame at 30 FPS. Detects lane center and computes pixel error $e(t)$ via image processing.

2

Controller

$\omega(t) = k_e \cdot e(t)$, clipped to $\pm\omega_{\max}$. Identical code runs in simulation and on the real robot.

3

diff_drive_ik

Converts (v, ω) to (v_L, v_R) . Same function used in simulation and on the real robot.

4

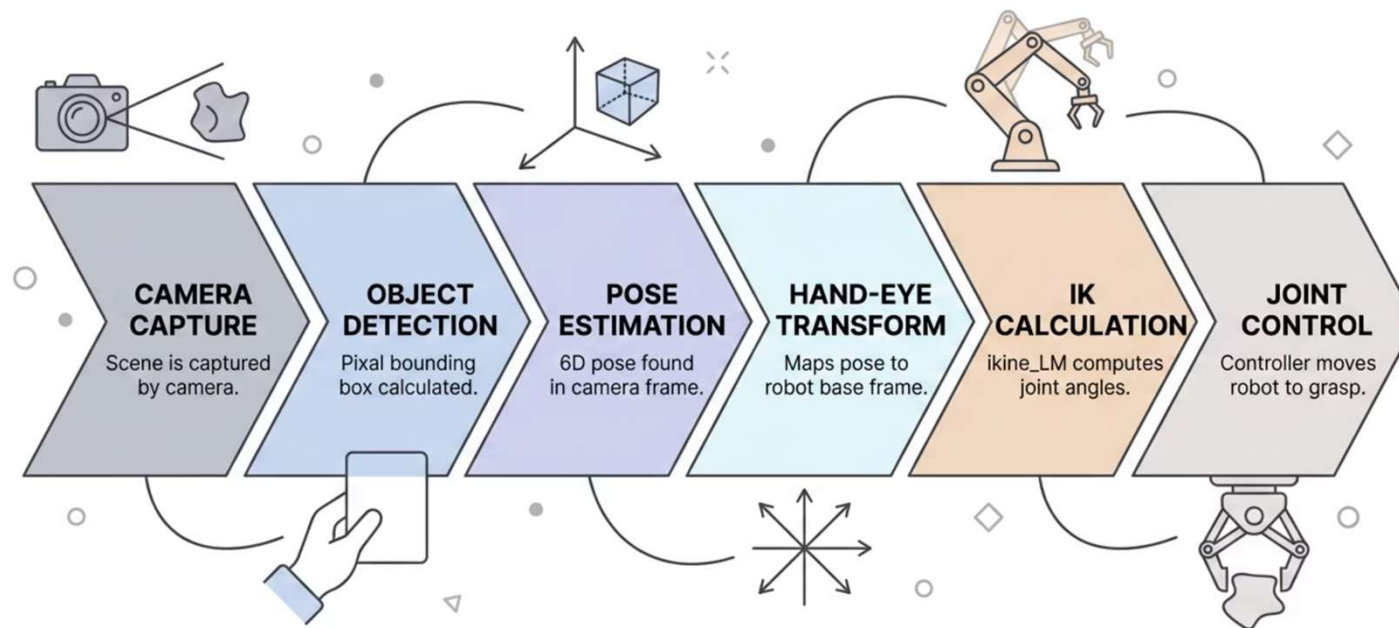
Motors

Wheel speed commands sent to hardware via ROS topic or SDK. Robot drives along the line.

The pipeline is **identical** between simulation and hardware — only the camera input source changes. This validates the simulation workflow for real deployment (Week 11)



From Perception to Action: Complete Flow



IK in the Robot Control Architecture

Where IK Lives in the Stack

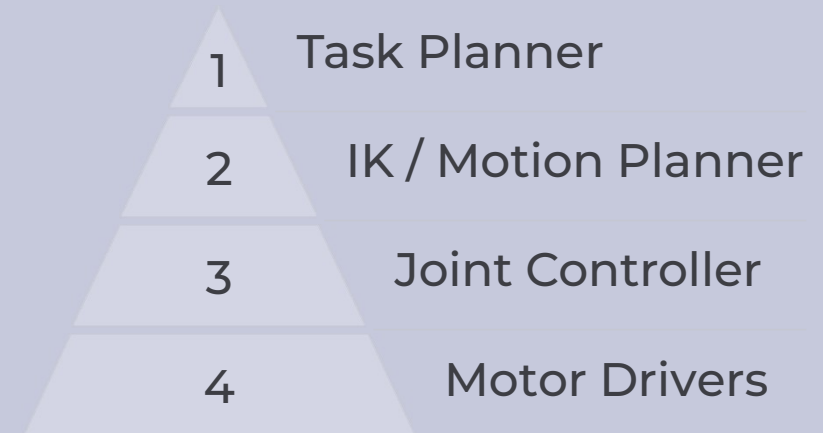
IK typically runs at the **motion planning layer**, generating a joint-space trajectory $\mathbf{q}(t)$ from a task-space trajectory $\mathbf{T}(t)$. The joint-space controller (PID, feedforward) then tracks $\mathbf{q}(t)$ at a faster rate (1 kHz vs. 10 Hz for IK).

Resolved-Rate Control

For tracking **moving targets** in real time, resolved-rate IK applies the Jacobian pseudoinverse at every control cycle:

$$\dot{\mathbf{q}} = \mathbf{J}^+(\mathbf{q}) \dot{\mathbf{x}}_{desired}$$

Control Hierarchy



Common IK Mistakes & How to Avoid Them

Forgetting Joint Limits & Collision Checks

IK solvers typically do not check collision or joint limit compliance automatically. Always post-validate `q_sol` against `robot.qlim` and run a collision check before executing on hardware.

Jumping Between Configurations

Solving IK only for the endpoint and commanding it directly causes the robot to take the shortest path in **joint space** — which may involve large, jerky movements or pass through obstacles in task space. Always interpolate.

Skipping FK Verification

An IK solver that "succeeds" but has high residual error can cause the robot to miss the target by centimeters. Always verify `T_check = fkine(q_sol)` and check $\|T_{check} - T_{des}\|$ before executing.

Debugging IK Problems — A Systematic Approach



Start Simple

Begin with a target very close to the current pose (translate 1 cm). Confirm IK works there before testing harder targets.



Try Multiple q_0

If convergence fails, try `robot.qr`, `q_cur`, and several random configurations. Different starting points explore different basins of attraction.



Print Everything

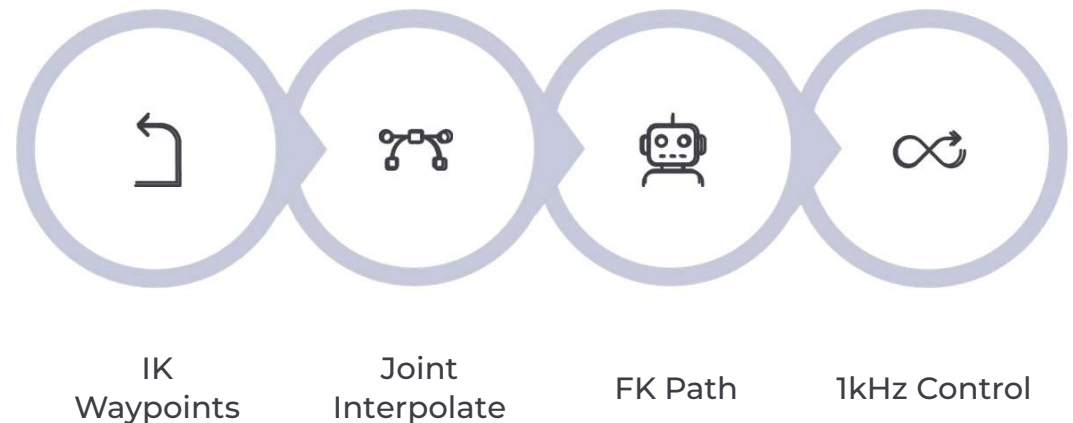
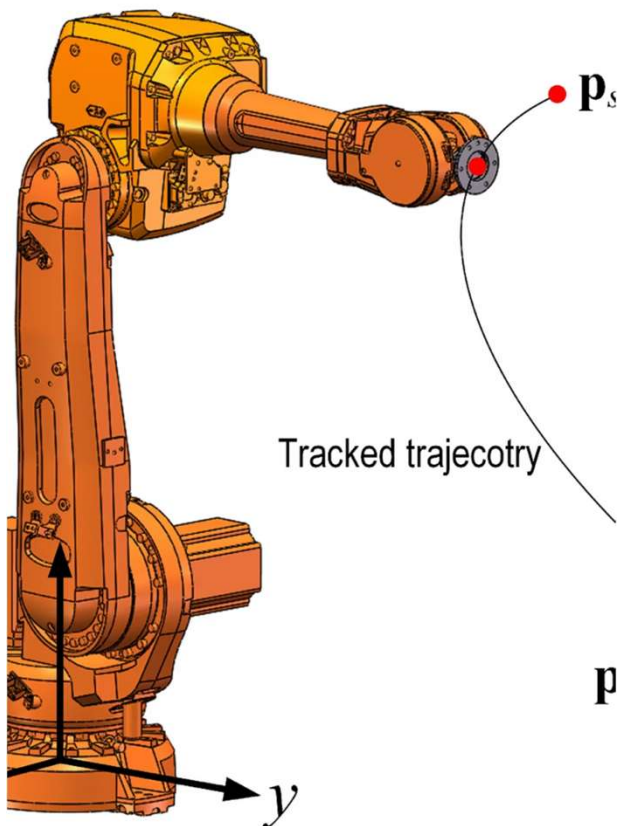
Always print `Tdes`, `Tcheck`, `||error||`, `sol.success`, and `sol.iterations`. Numerical debugging requires visibility into the solver's state.



Tune Solver Parameters

Adjust the damping factor λ (increase for stability near singularities) and the maximum iteration count. The Toolbox exposes these as keyword arguments.

From IK Solutions to Smooth Trajectories



IK at discrete waypoints is just the starting point. A complete trajectory planner interpolates smoothly between waypoints in joint space (cubic splines, quintic polynomials) to ensure continuous position, velocity, and optionally acceleration profiles — preventing mechanical stress and ensuring smooth task execution.

Frequently Asked Questions

→ Can IK handle moving targets in real time?

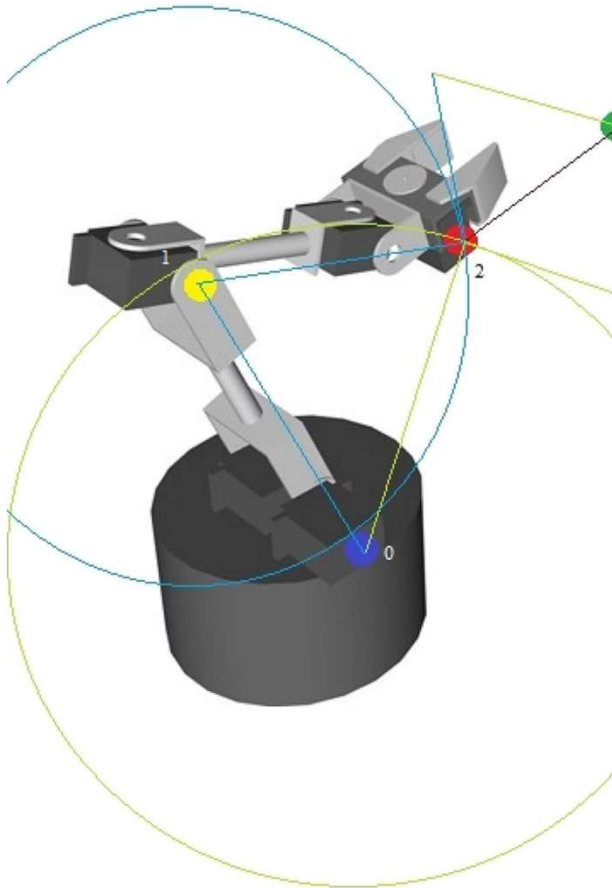
Yes — resolved-rate control applies the Jacobian pseudoinverse at every control cycle (e.g., 100 Hz) to track a continuously moving target. The target must move slower than the IK can converge, which limits the maximum tracking speed.

→ What if the robot has obstacles to avoid?

Standard IK ignores obstacles. Collision-aware IK (constrained IK or optimization-based motion planning like RRT/CHOMP) is needed for obstacle avoidance — this is covered in motion planning modules beyond this course.

→ Why does `ikine_LM` sometimes give a different solution than expected?

Numerical IK converges to the local minimum nearest to q_0 . Changing q_0 changes which solution is found. This is by design — use q_{cur} as q_0 to get the solution closest to the robot's current state.



Week 8 Summary — Part 1: Arm IK

The IK Problem

Given desired end-effector pose $T_{des} \in SE(3)$, find joint configuration q such that $FK(q) \approx T_{des}$. Always verify with FK after solving.

Analytic IK (2R)

Law of cosines gives θ_2 ; atan2 geometry gives θ_1 . Up to 2 solutions (elbow-up / elbow-down). Builds geometric intuition for all IK.

Numerical IK (6-DOF)

Levenberg–Marquardt (`ikine_LM`) iteratively reduces pose error. Initial guess q_0 determines which solution is found. Check `sol.success` and `residual`.

Week 8 Summary — Part 2: Mobile IK & Labs

"IK" for Mobile Robots

Converting desired body motion (v , ω) to wheel commands (v_L , v_R) is the mobile robot's IK — a simple linear inversion with a unique solution. Line following adds a perception layer: image error $\rightarrow \omega \rightarrow$ wheel commands.

Lab Highlights

- **8.1:** Built 2R analytic IK from scratch
- **8.2–8.4:** Explored numerical IK convergence, workspace limits, and initial guess effects
- **8.5–8.6:** Executed trajectories and selected optimal solutions
- **8.M1–M4:** Simulated mobile IK and line following pipeline

Core Takeaways

- Always verify IK solutions with FK before commanding hardware
- Use q_{cur} as initial guess for smooth motion
- Check `sol.success`, residual, and joint limits
- IK is the bridge from perception/planning to motor control

Preparing for Week 9 — Vision-Guided Grasping

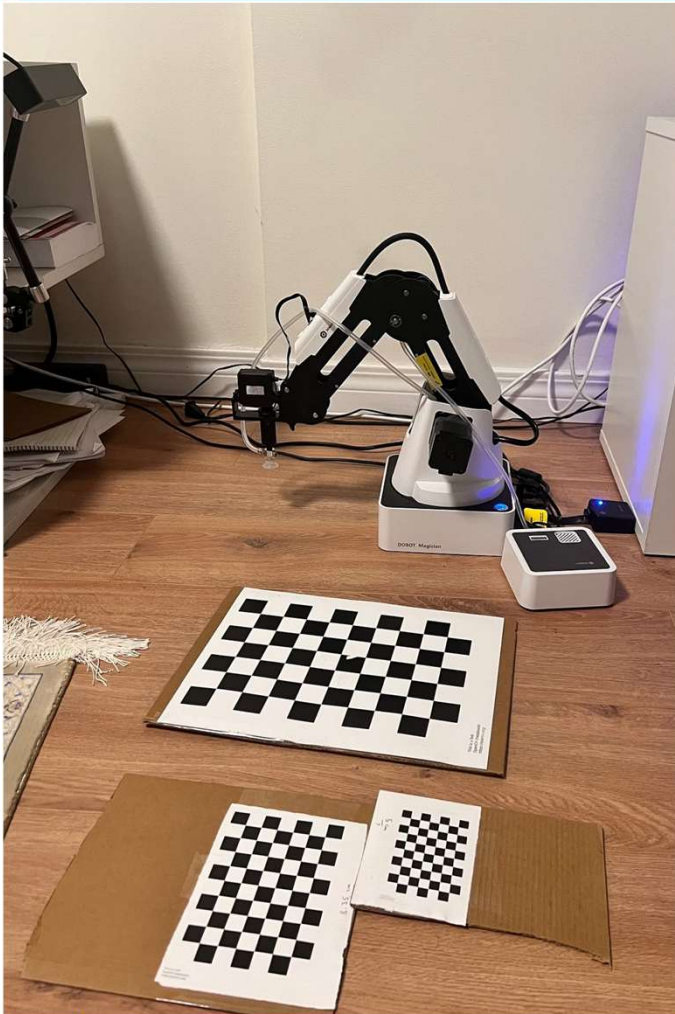
Week 9 Preview

Next week integrates everything from Weeks 6–8 into a complete pick-and-place pipeline:

- **Hand-eye calibration:** determine $T_{\text{camera} \rightarrow \text{base}}$
- **Object pose estimation:** $A_I \rightarrow T_{\text{obj}}$ in camera frame
- **Coordinate mapping:** $T_{\text{obj}} \rightarrow T_{\text{target}}$ in base frame
- **IK (Week 8):** $T_{\text{target}} \rightarrow q \rightarrow \text{arm}$ executes grasp

Your IK Code Is the Engine

The `ikine_LM`-based pipeline you built this week becomes the motion execution backend for Week 9's vision-guided grasping demo. Make sure your verification workflow (FK check, joint limit check, convergence check) is solid — it will run on real hardware next week.



1

Hand-Eye Calibration

Determine the fixed transform between camera frame and robot end-effector/base

2

Camera → Base Transform

Map detected object pose from camera frame into robot base coordinates

3

IK (This Week!)

$T_{\text{target}} \rightarrow \text{ikine_LM} \rightarrow \text{joint angles } q$

4

Autonomous Grasp

AI detects object → robot moves → picks it up

Everything from Weeks 5–8 converges in Week 9. Pose estimation, frame transforms, FK, and IK all combine into a single autonomous grasping pipeline running on your physical robot.

Looking Ahead — The Complete Manipulation Stack

1

Week 6–7

SE(3), PoE, Jacobian, Forward Kinematics — the mathematical foundation.

2

Week 8

Inverse Kinematics — analytic (2R) and numerical (6-DOF). The engine of manipulation.

3

Week 9

Hand-eye calibration + coordinate mapping. AI → Ttarget → IK → grasp.

4

Week 11

Real camera → real error → real wheel commands. Line following on JetRacer with live video.

5

Final Project

End-to-end: detect → map → IK → pick → place. Full vision-guided robotic manipulation.

Q&A — Inverse Kinematics

Questions, clarifications, and ideas for your final project.

Discussion Prompts

- When would you choose analytic IK over numerical IK in a real deployment?
- How would you handle a case where the desired grasp pose is outside the workspace?
- What additional constraints would you add to `select_best_ik()` for a real Yahboom task?

Final Project Ideas

- Combine Week 9 hand-eye calibration with this week's IK pipeline for a full pick-and-place demo
- Implement resolved-rate control for a moving target using continuous IK
- Build a workspace visualizer that color-codes regions by IK solution count and manipulability

AI407DE01

WEEK 8 COMPLETE

